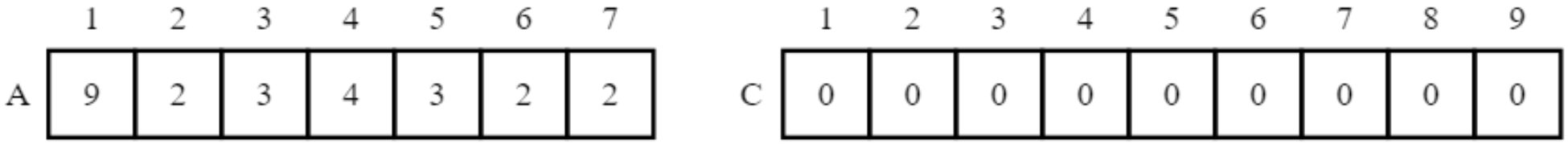
Sorting

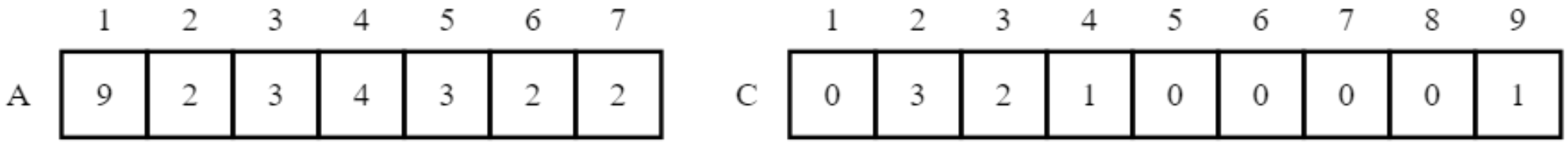
Counting Sort

- Suppose we know that the list to be sorted consists of n elements in the range 1 to k
- We can allocate a counting array C of size k and initialize this to all zeroes:



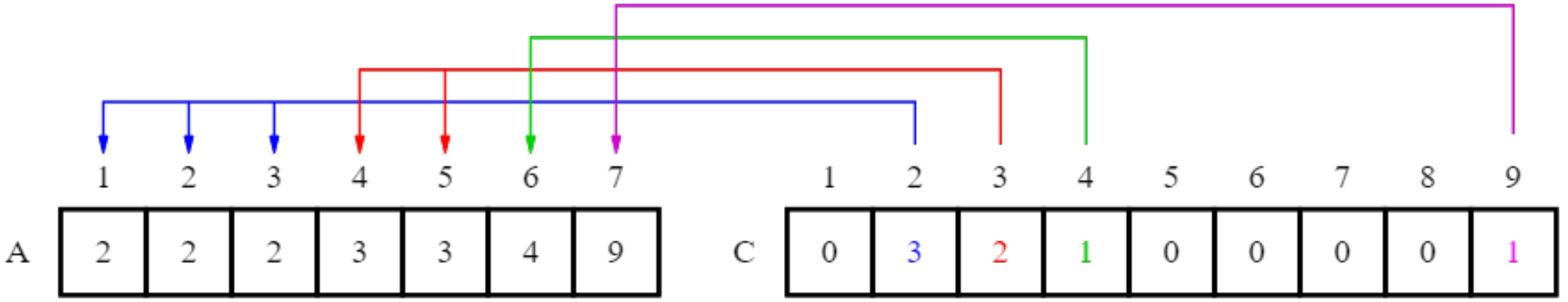
Counting Sort

- We then run through A, and for each element A[i], increment the value in C[A[i]] as illustrated in the following diagram:



Counting Sort

- Once this process is complete, each C[i] contains the number of times i occurs in A. We can now copy the values from C back into the array A, ensuring that i is copied into A[i], C[i] times in succession, as shown



Complexity Calculation

- n: Number of elements in the input array
- k: Range (max-min+1)

- Find the range of values: O(n)
  - Traverse the input array once to find max and min
- Initialize count array: O(k)
  - Allocate and initialize an array of size k+1
- Count occurrences: O(n)
  - Traverse the input array and update the count array
- Accumulate counts: O(k)
  - Traverse the count array for cumulative counts
- Sorted output: O(n)
  - Traverse the input array using count array for indexing

Total Time Complexity

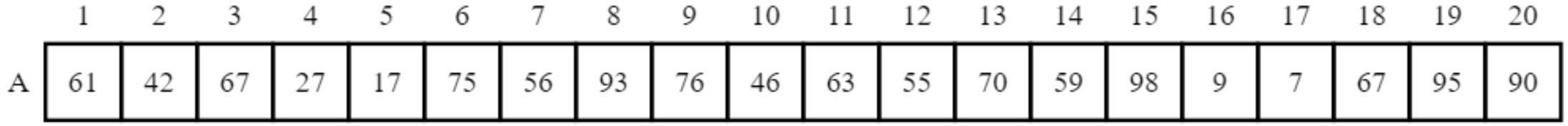
$$T(n,k) = O(n)+O(k)+O(n)+O(k)+O(n)$$
$$= O(n+k)$$

Limitations

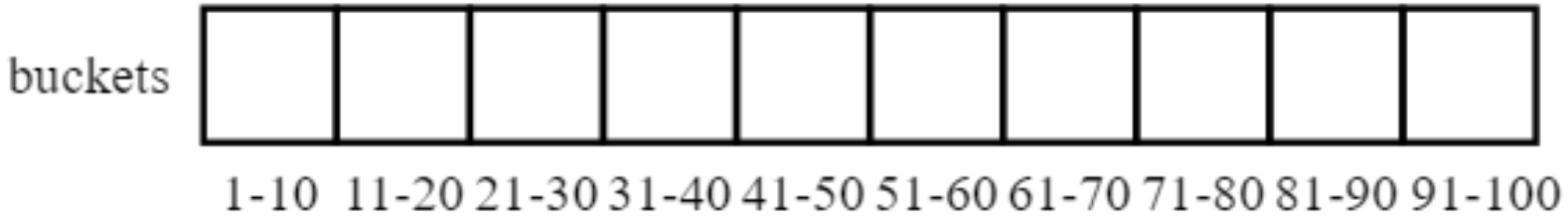
- it can only work for integer arrays
- it can require a huge counting array C. For example, A= {4,5,1,7,1000000} would require C to be of size 1000000, just to sort a five-element array
- Time complexity: O(n+k). For a large k this method performs worse than insertion or selection sort (e.g. when $k=n^3$)

Bucket Sort

- This algorithm works well if the numbers of the input array are uniformly distributed over a range, say [1,k]. Let us take an example of a size 20 array whose elements are distributed over the range [1,100]



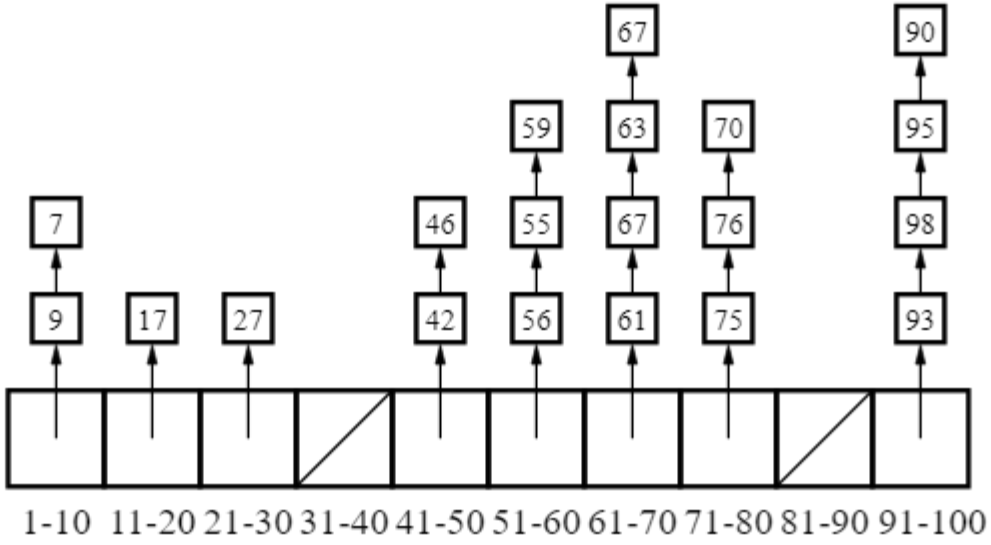
- We can now create ten buckets to hold the elements of the array. These buckets hold elements 1-10, 11-20, … , 91-100 as shown



Bucket Sort

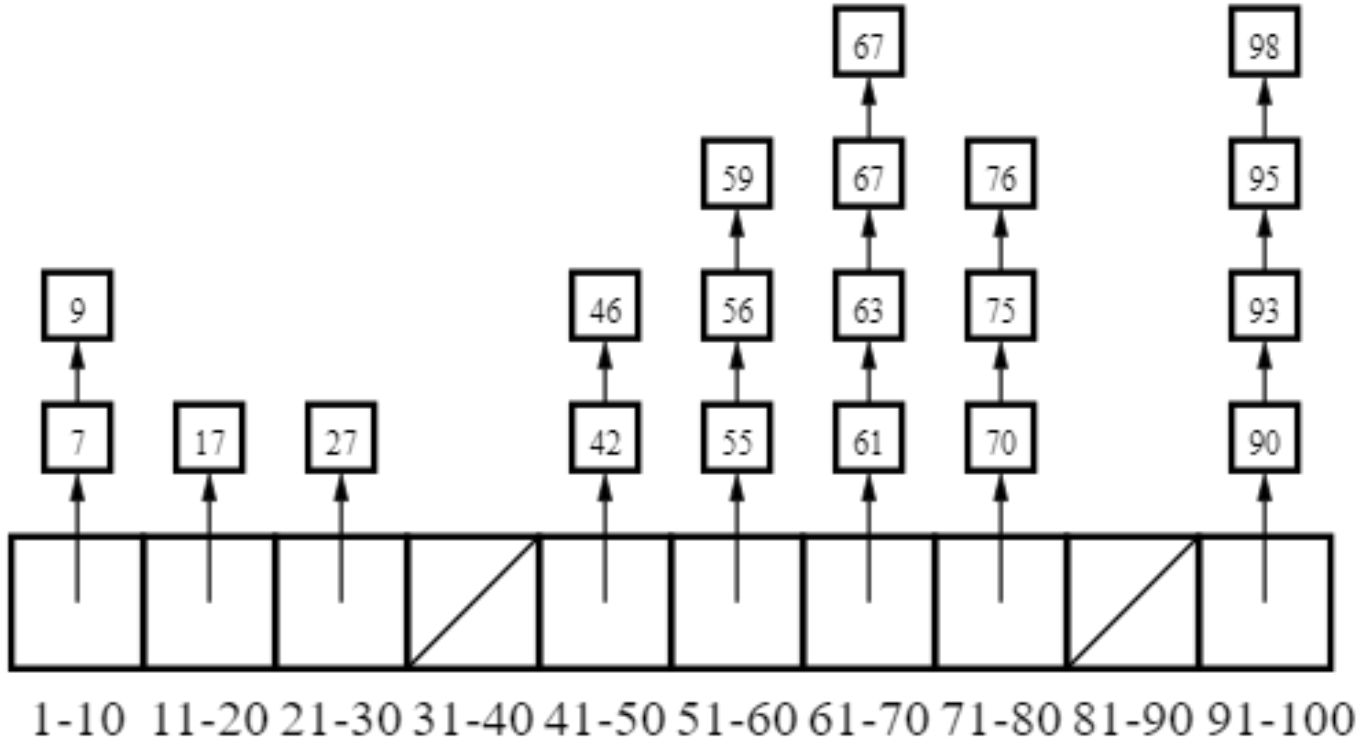
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	61	42	67	27	17	75	56	93	76	46	63	55	70	59	98	9	7	67	95	90

- Since each bucket holds a variable number of elements, we associate a linked list with each bucket. The order of the elements is the order in which the elements 51 ≤ 60 are encountered when sweeping array A from left to right. Buckets marked with a diagonal line mean there are no elements in A that fall into the corresponding range



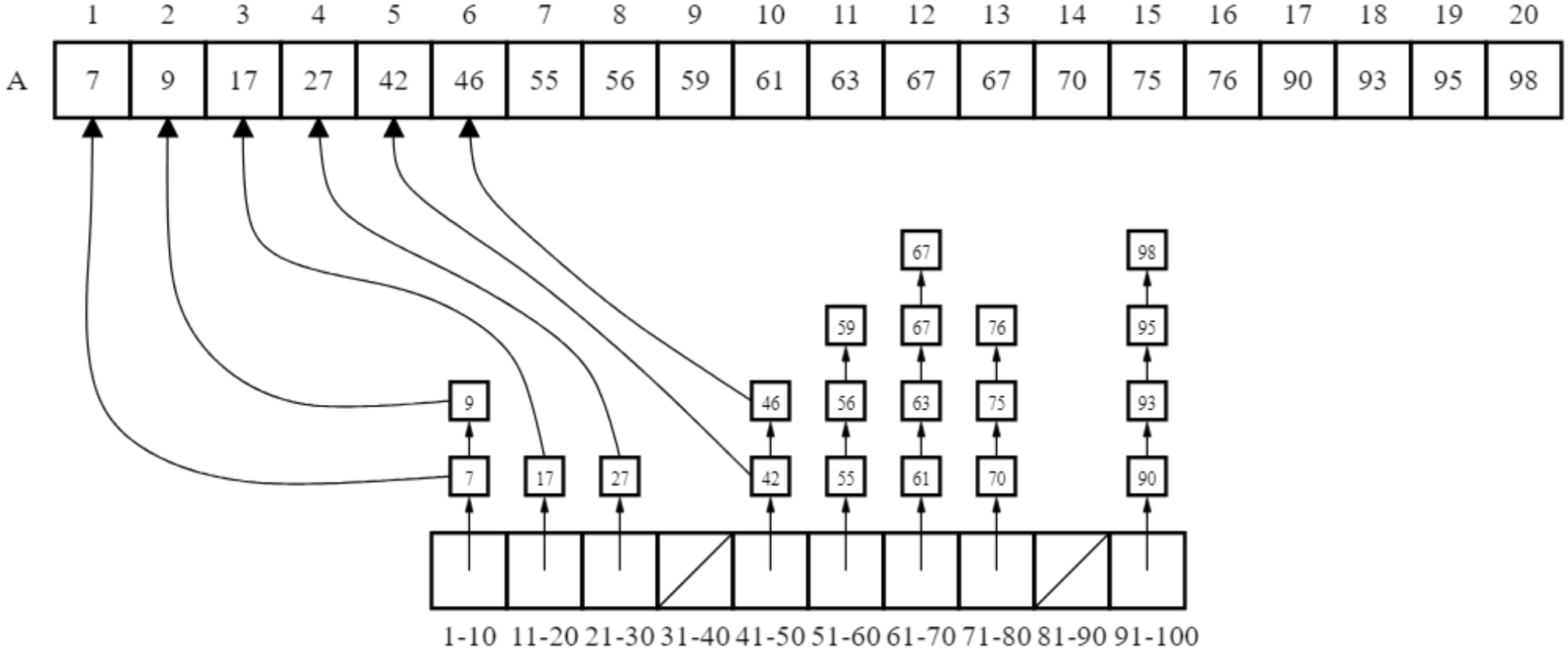
Bucket Sort

- We can now sort the individual linked lists using insertion sort. This results in:



Bucket Sort

- We can now step through the buckets, copying the elements of each linked list into the original array A. This results in a sorted array A:



Complexity Calculation

Total Time Complexity
$T(n,k) = O(n) + O(n^2/k) + O(n)$

- Distribute elements into buckets: O(n)
  - Calculate bucket index for each element and assign
- Sort each bucket: $O(n^2/k)$
  - Say, we use Insertion Sort. There are n/k elements in a bucket

$$\sum_{i=1}^{k} O\left(\frac{n}{k}\right)^2 = k \cdot O\left(\frac{n}{k}\right)^2 = O\left(\frac{n^2}{k}\right)$$

- Concatenate buckets: O(n)
  - Visit each element in the bucket and place in the sorted array

Distribute Elements

- Normalize the data so that they are in the range [0,1]

  Array=[0.42,0.32,0.53,0.37,0.91,0.25,0.88]

- Let, k = 10

- Bucket assignment:
  - Bucket Index=floor(x×k)

For 0.42: Bucket Index=floor(0.42×10)=floor(4.2)=4

For 0.32: Bucket Index=floor(0.32×10)=floor(3.2)=3

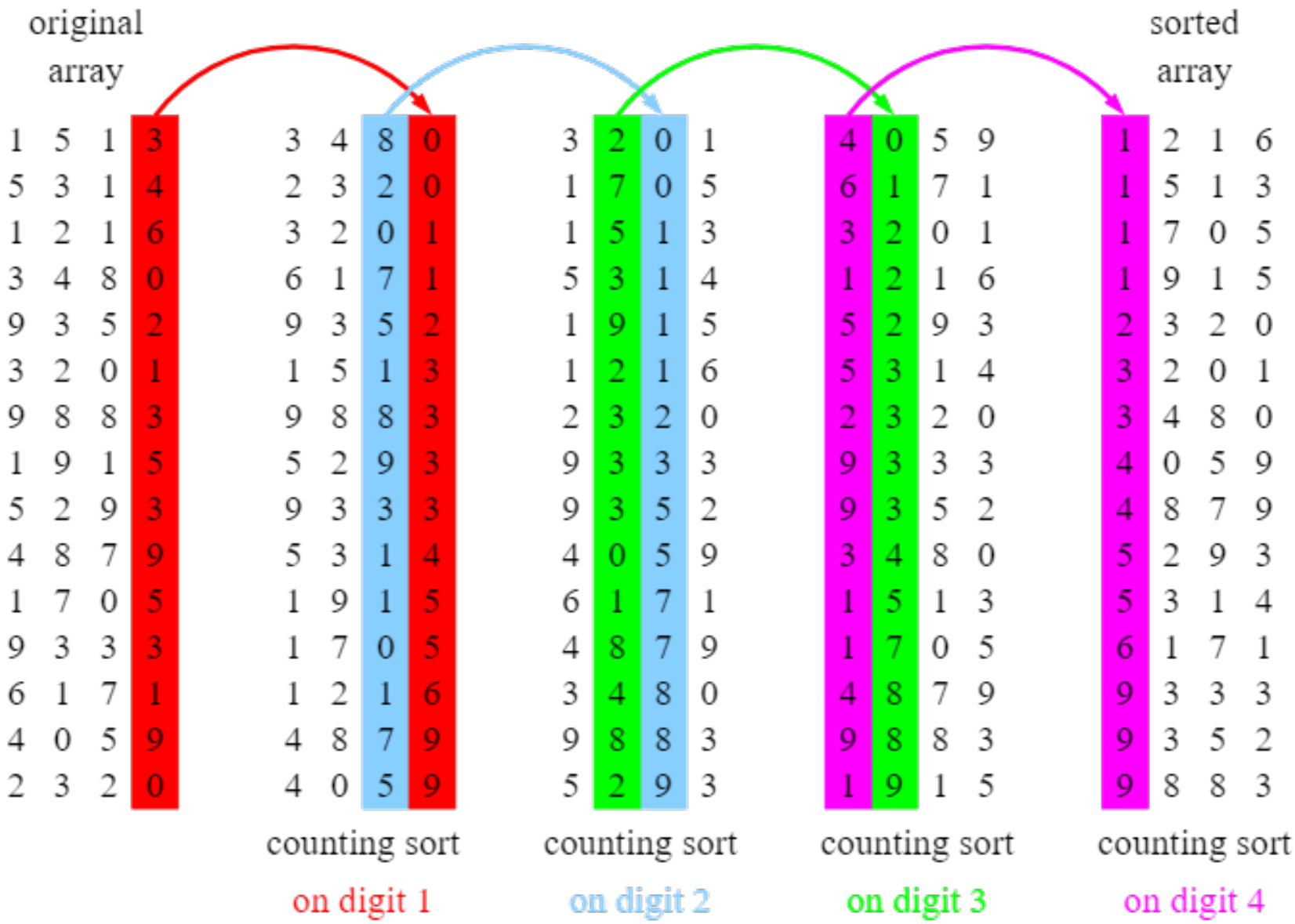
Radix Sort

- In the case of Counting sort, we required time O(n+k), but k was as large as the maximum number in the array to be sorted. This led to space and time complexity of O(n+k). For Bucket sort, we had a small number of buckets, but the linked list in a bucket could be as large as n, leading to $O(n^2)$ runtime since insertion sort was required to sort the linked lists

- Radix Sort uses a fixed number of buckets and repeatedly sorts the digits in each number in the array

Radix Sort

- Each number in the array has four digits
- Start with the least significant digit
- Sort using this digit only
- Repeat for other digits



Complexity Calculation

- n: Number of elements to be sorted
- d: Number of digits of the largest number, $\log_2(M)$

- Sorting each digit
  - Use counting sort: O(n)
  - Since there are ‘d’ digits and d = $\log_2(M)$, O(n $\log_2(M)$)

Parallel Sort

- Best sequential sorting algorithms (mergesort and quicksort) have (respectively worst-case and average) time complexity of O(n log(n))

- The best we can aim with a parallel sorting algorithm using n processing units is thus a time complexity of

$$O(n \log(n))/n = O(\log(n))$$

Compare-and-Exchange

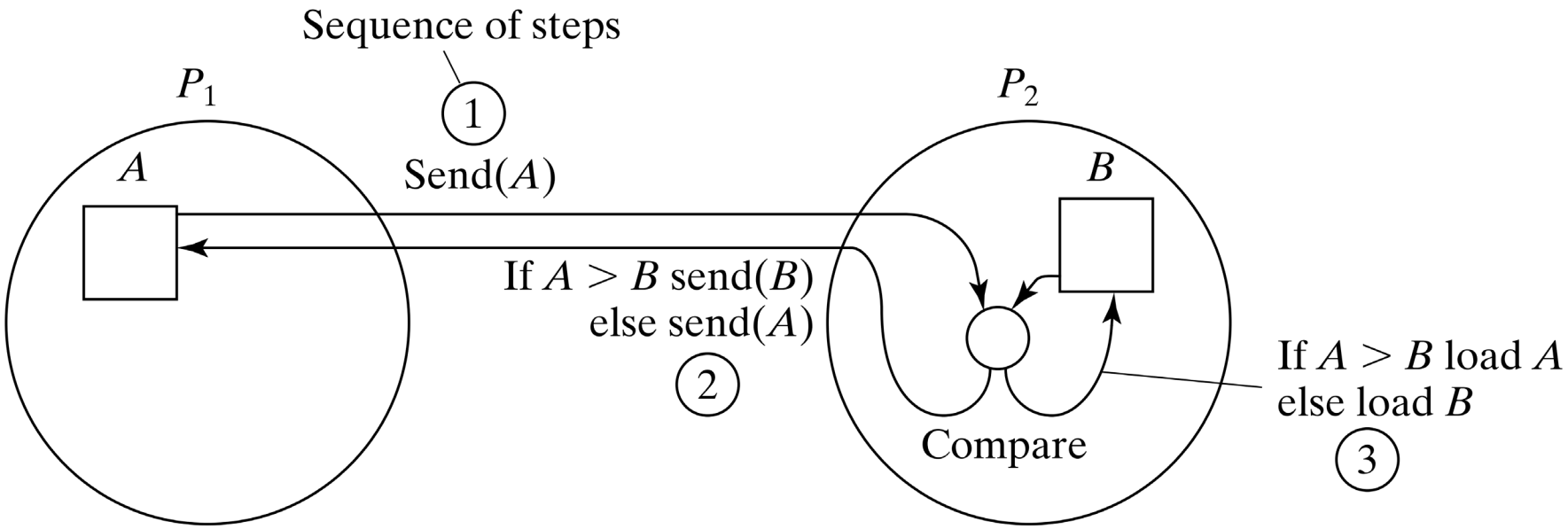
- An operation that forms the basis of several classical sequential sorting algorithms is the compare-and-exchange (or compare-and-swap) operation
- In a compare-and-exchange operation, two numbers, say A and B, are compared and if they are not ordered, they are exchanged. Otherwise, they remain unchanged

```
if (A > B) { // sorting in increasing order
  temp = A;
  A = B;
  B = temp;
}
```

Question: how can compare-and-exchange be done in parallel?

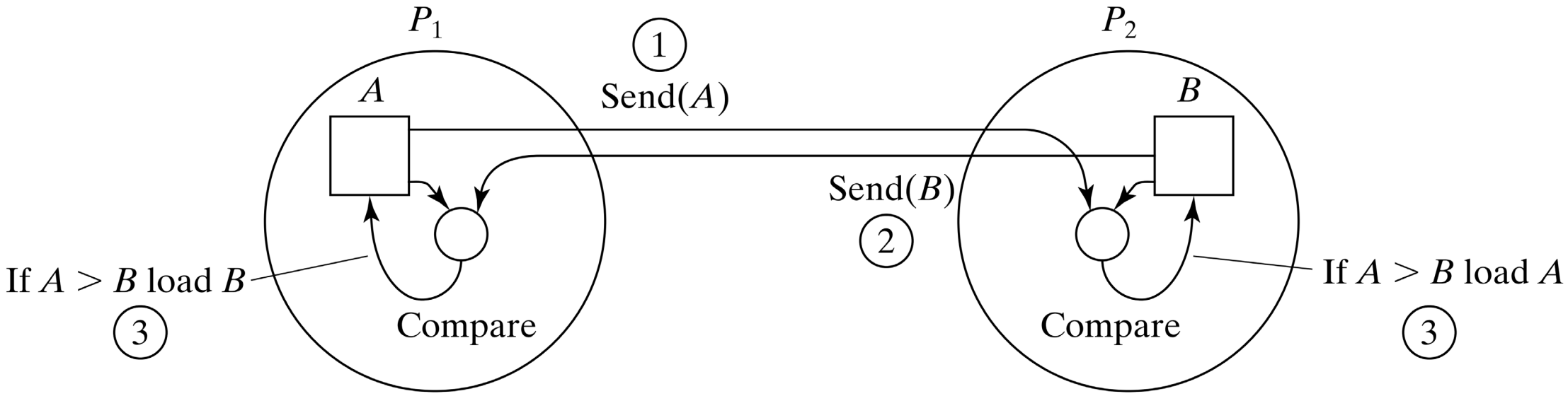
Parallel Compare-and-Exchange

- **Version 1** – P1 sends A to P2, which then compares A and B and sends back to P1 the min(A, B)



Parallel Compare-and-Exchange

- **Version 2** – P1 sends A to P2 and P2 sends B to P1, then both perform comparisons and P1 keeps the min(A, B) and P2 keeps the max(A, B)



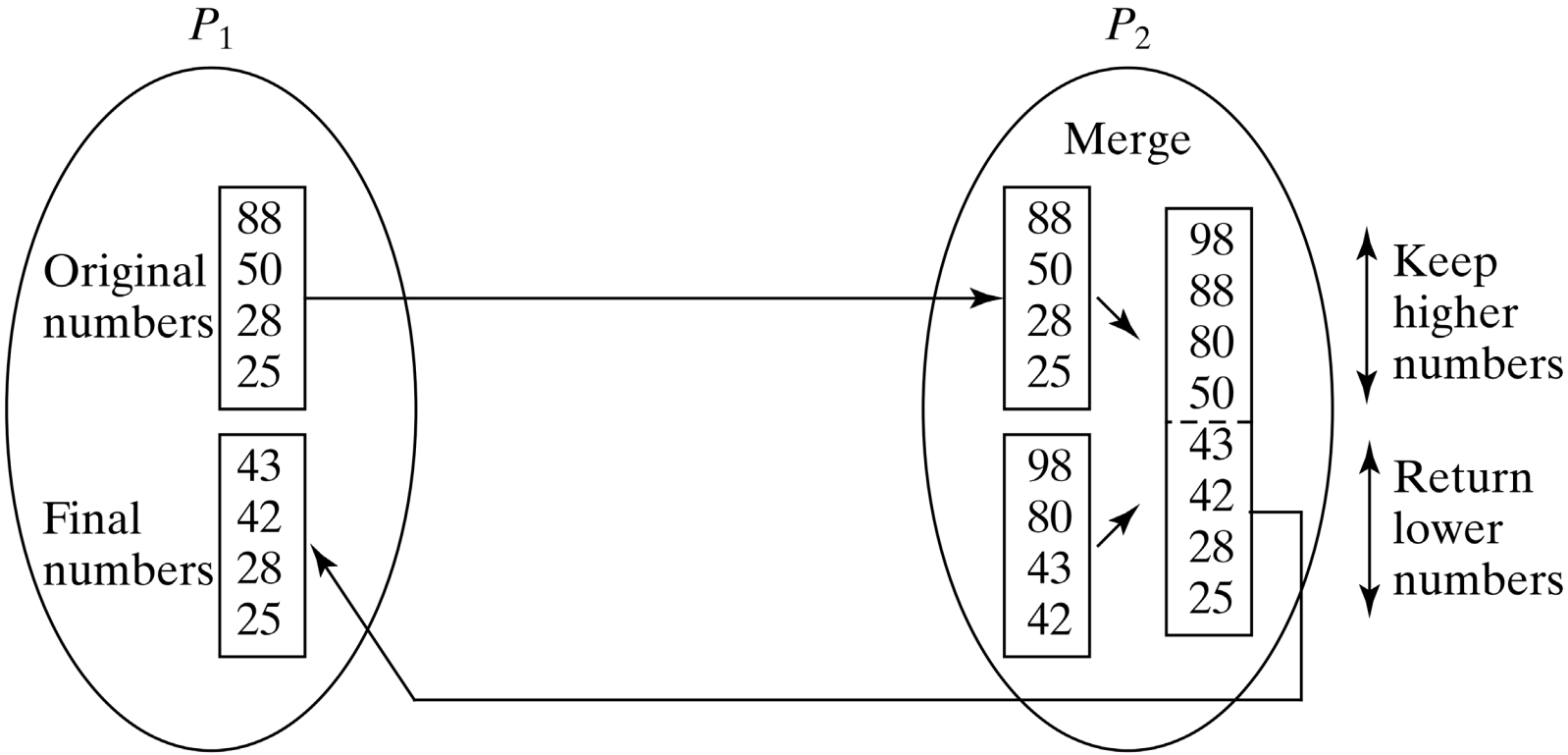
Data Partitioning

- So far, we have assumed that there is one processing unit for each number, but normally there would be many more numbers (n) than processing units (p):
  - in such cases, a list of n/p numbers would be assigned to each processing unit.

- When dealing with lists of numbers, the operation of merging two sorted lists is a common operation in sorting algorithms

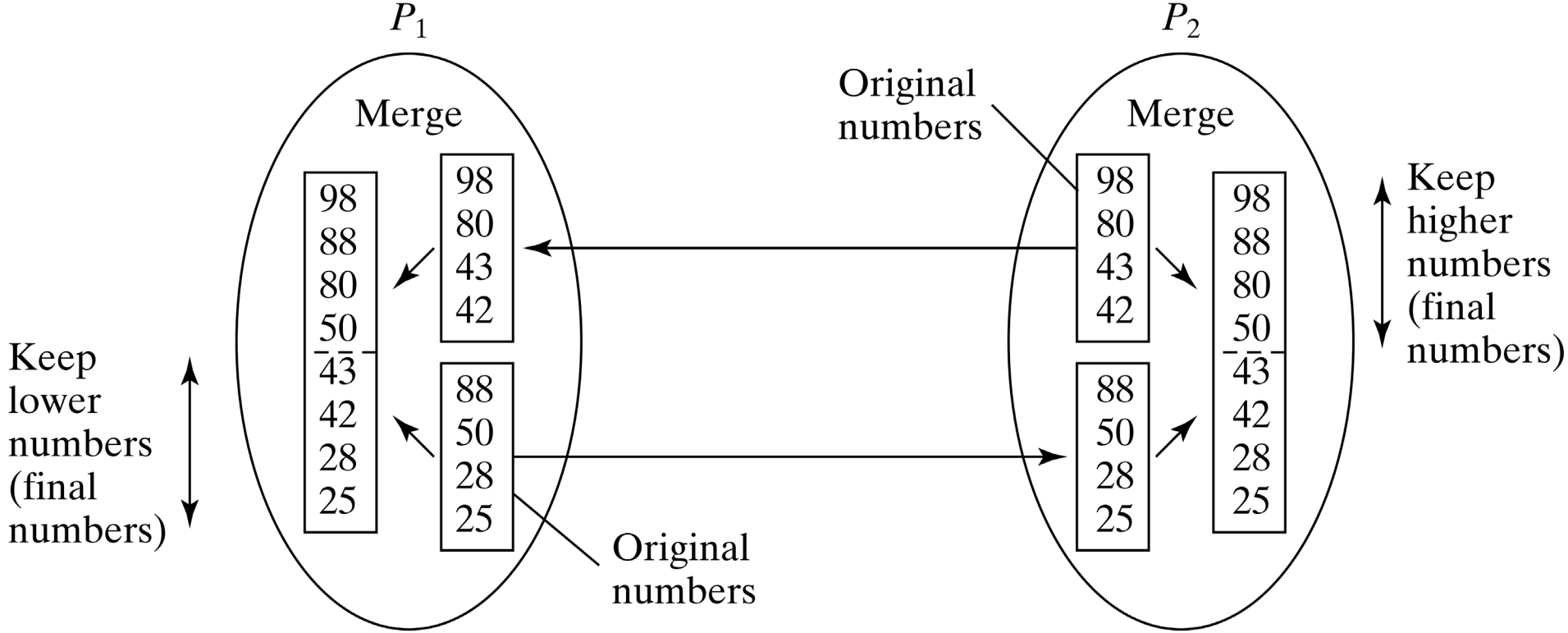
Parallel Merging

- Version 1 – P1 sends its list to P2, which then performs the merge operation and sends back to P1 the lower half of the merged list



Parallel Merging

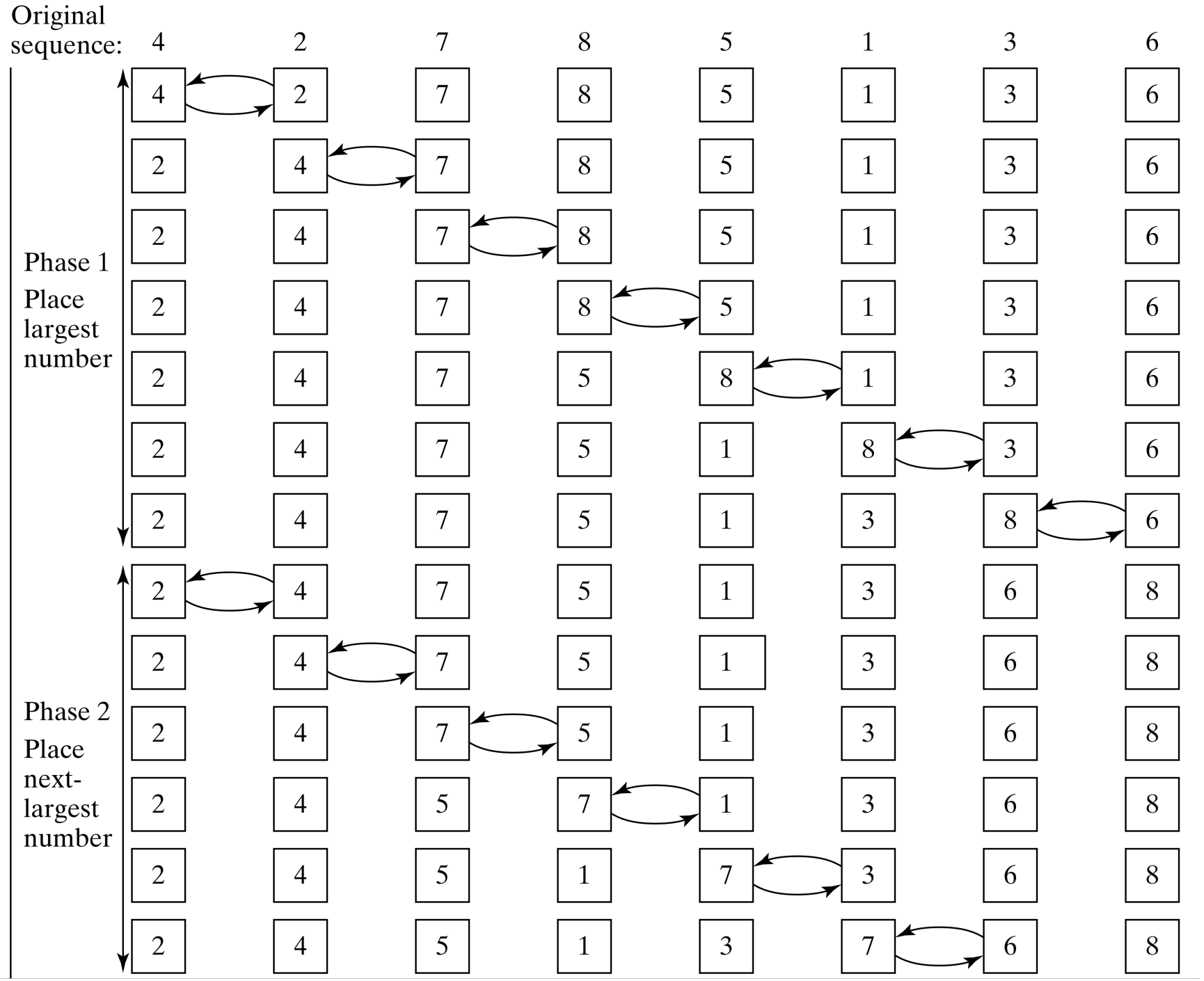
- Version 2 – both processing units exchange their lists, then both perform the merge operation and P1 keeps the lower half of the merged list



Bubble Sort: Compare-and-Exchange

- In bubble sort, the largest number is first moved to the very end of the list by a series of compare-and-exchange operations. The procedure repeats, stopping just before the previously positioned largest number, to get the next-largest number. In this way, the larger numbers move (like a bubble) toward the end of the list

```
for (i = N - 1; i > 0; i--)
   for (j = 0; j < i; j++) {
          k = j + 1;
          if (a[j] > a[k]) {
             temp = a[j];
             a[j] = a[k];
             a[k] = temp;
          }
   }
```


Original sequence:
4 2 7 8 5 1 3 6
Phase 1
Place largest number
Phase 2
Place next-largest number

Complexity Calculation

- Outer Loop: The algorithm makes n−1 passes through the array

- Inner Loop: On the i-th pass (starting from i=0), the inner loop runs n−1−i comparisons

Total comparisons = (n−1)+(n−2)+⋯+1

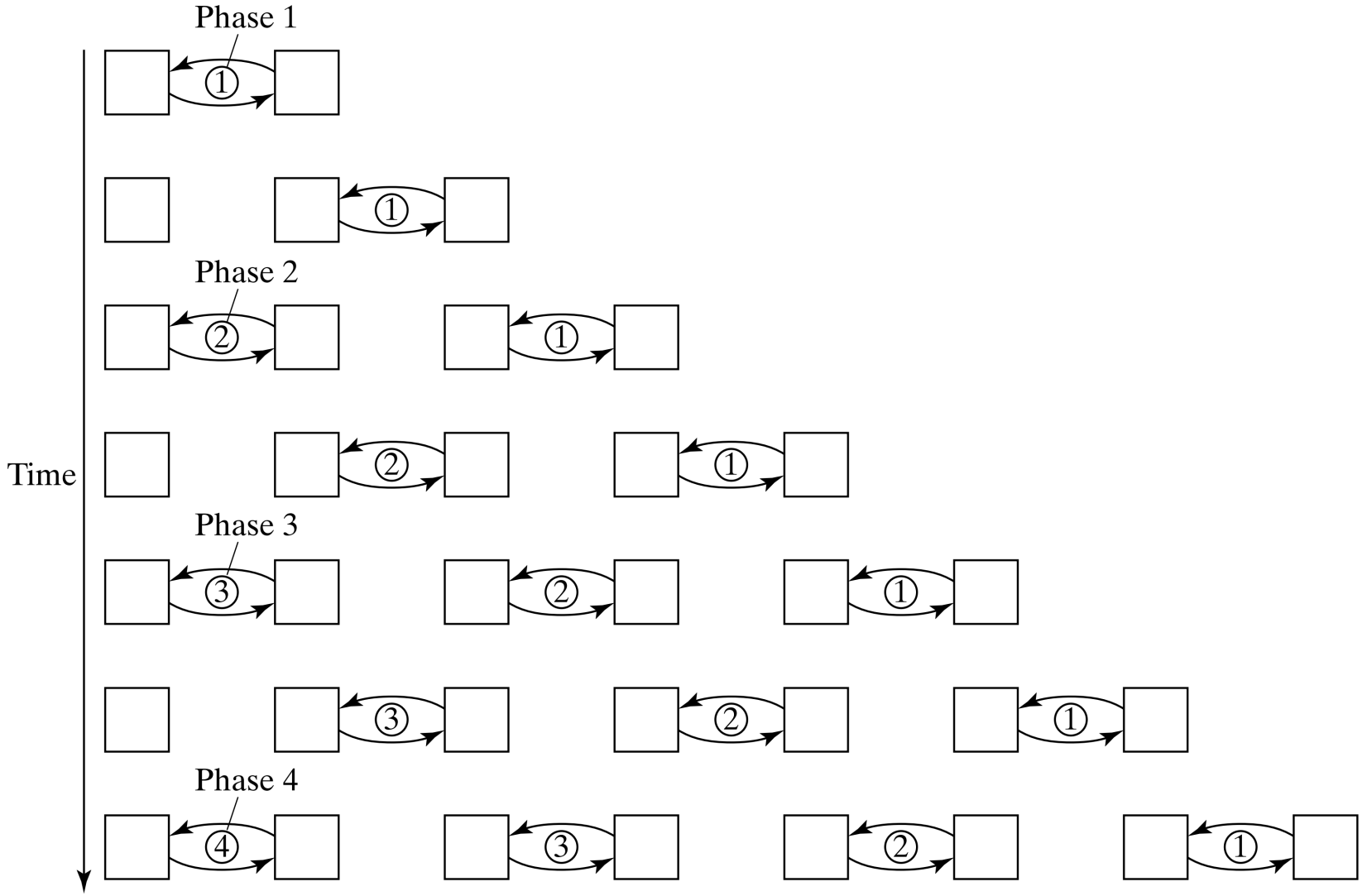
= n(n-1)/2

- $O(n^2)$

```
for (i = N - 1; i > 0; i--)
   for (j = 0; j < i; j++) {
         k = j + 1;
         if (a[j] > a[k]) {
            temp = a[j];
            a[j] = a[k];
            a[k] = temp;
         }
   }
```

Parallel Bubble Sort

- A possible idea is to run multiple iterations in a pipeline fashion, i.e., start the bubbling action of the next iteration before the preceding iteration has finished in such a way that it does not overtake it



Odd-Even Transposition Sort

- Variant of bubble sort which operates in two alternating phases:
- Even Phase: even processes exchange values with right neighbors

  (P0 $\leftrightarrow$ P1, P2 $\leftrightarrow$ P3, ...)
- Odd Phase: odd processes exchange values with right neighbors

  (P1 $\leftrightarrow$ P2, P3 $\leftrightarrow$ P4, ...)

- For sequential programming, odd-even transposition sort has no particular advantage over normal bubble sort. However, its parallel implementation corresponds to a time complexity of O(n)

Odd-Even Transposition Sort

- In the even phase, we have the compare and exchanges: P0 ↔ P1, P2 ↔ P3, etc. Using the same form of compare and exchange as Version 2 , the code could be

$P_i,\ i = 0, 2, 4, \ldots, n-2$ (even)

```
recv(&A, P_{i+1});
send(&B, P_{i+1});
if (A < B) B = A;
```

$P_i,\ i = 1, 3, 5, \ldots, n-1$ (odd)

```
send(&A, P_{i-1});          /* even phase */
recv(&B, P_{i-1});
if (A < B) A = B;           /* exchange */
```

where the number stored in P_i (even) is B, and the number stored in P_i (odd) is A

Odd-Even Transposition Sort

- In the odd phase, we have the compare and exchanges: P1 ↔ P2, P3 ↔ P4, etc. This could be coded as

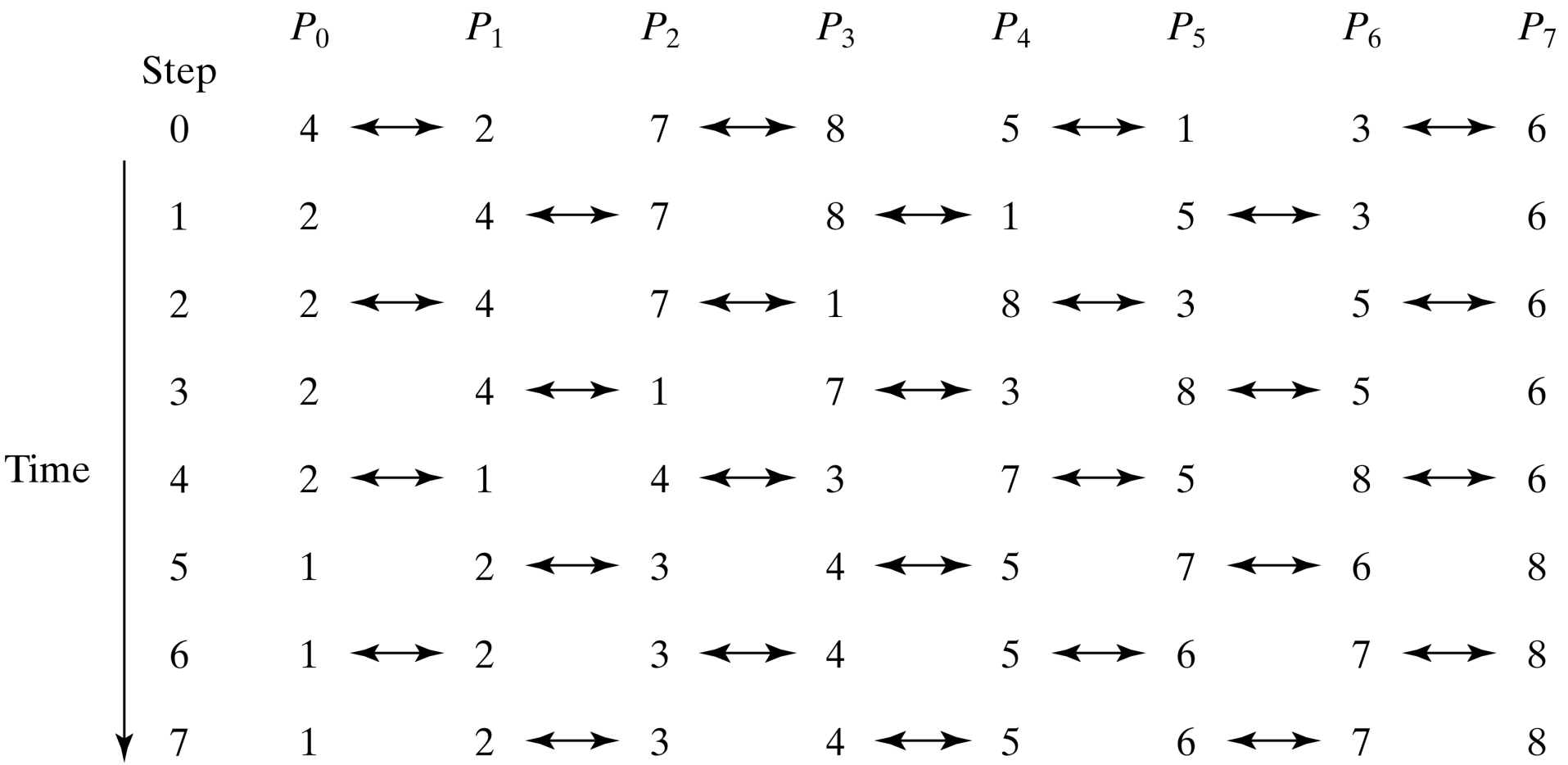
P_i, $i = 1, 3, 5, \ldots, n-3$ (odd)

```
send(&A, P_{i+1});
recv(&B, P_{i+1});
if (A > B) A = B;
```

P_i, $i = 2, 4, 6, \ldots, n-2$ (even)

```
recv(&A, P_{i-1});          /* odd phase */
send(&B, P_{i-1});
if (A > B) B = A;           /* exchange */
```

Odd-Even Transposition Sort



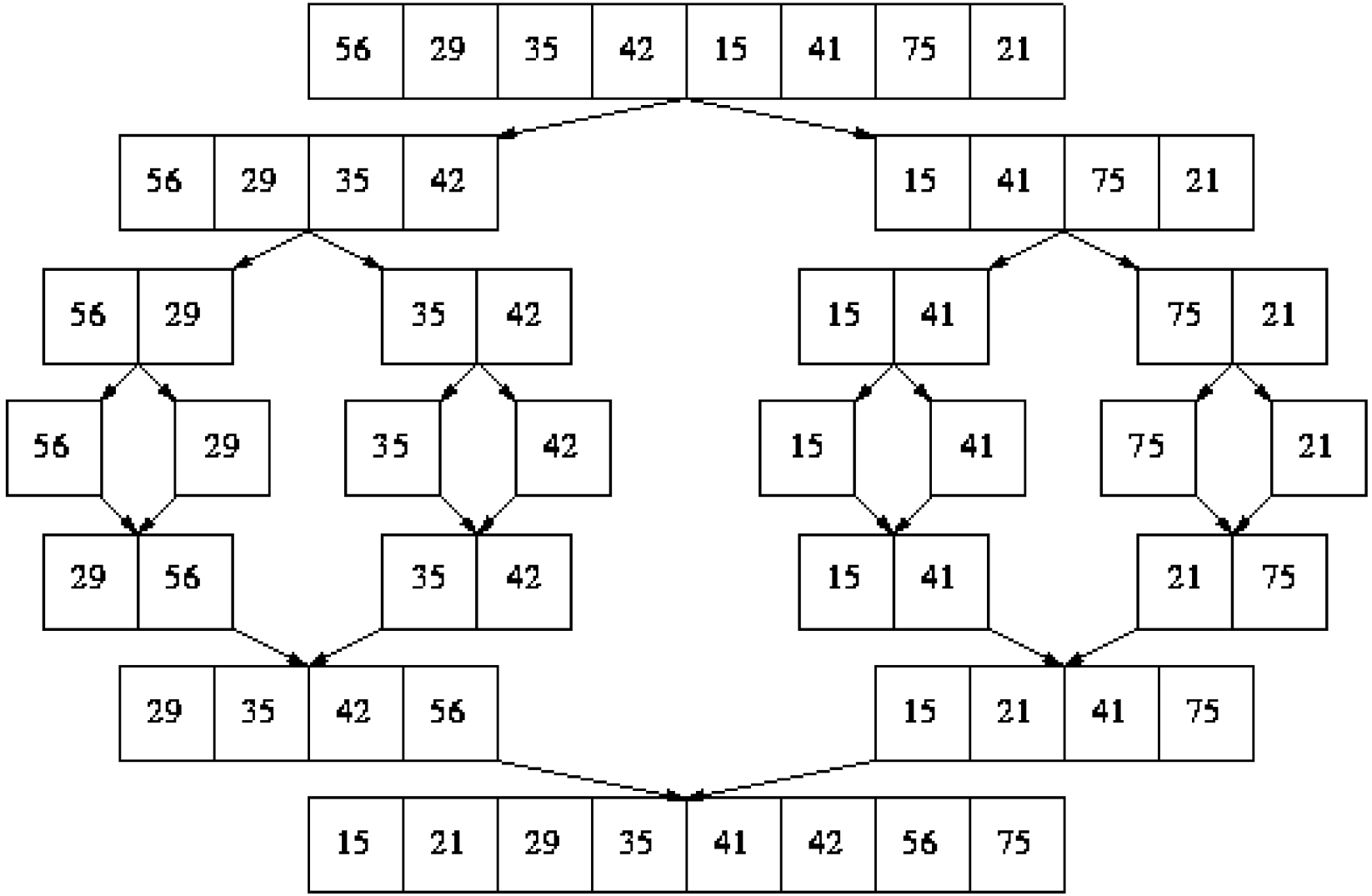
Mergesort

- Mergesort is a classical sorting algorithm using a divide-and-conquer approach

- The initial unsorted list is first divided in half, each half sublist is then applied the same division method until individual elements are obtained

- Pairs of adjacent elements/sublists are then merged into sorted sublists until the one fully merged and sorted list is obtained

Mergesort


56 29 35 42 15 41 75 21
56 29 35 42
15 41 75 21
56 29
35 42
15 41
75 21
56
29
35
42
15
41
75
21
29 56
35 42
15 41
21 75
29 35 42 56
15 21 41 75
15 21 29 35 41 42 56 75

Mergesort

Computations only occur when merging the sublists. In the worst case, it takes 2s − 1 steps to merge two sorted sublists of size s. If we have m = n/s sorted sublists in a merging step, it takes

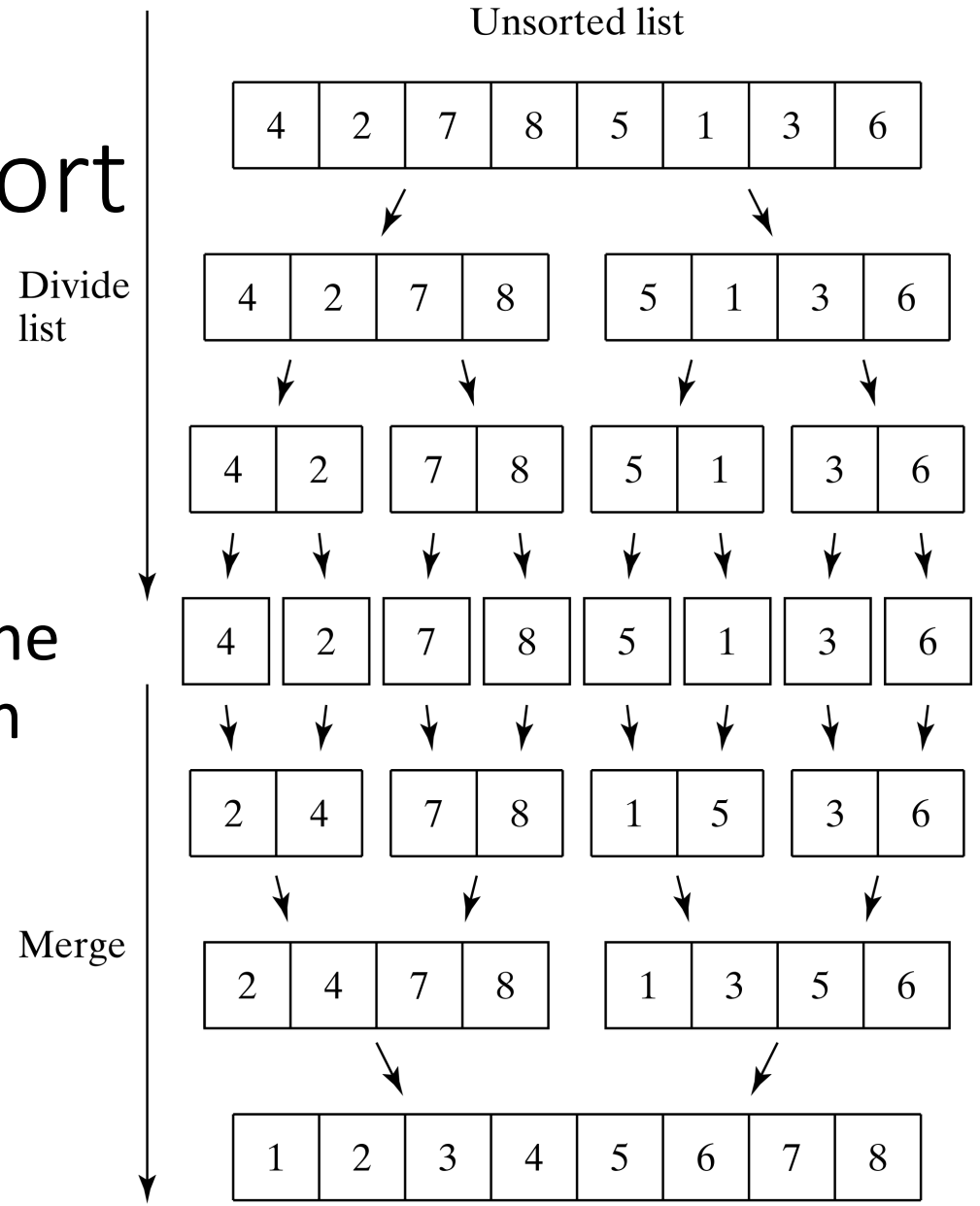
$$\frac{m}{2}(2s - 1) = ms - \frac{m}{2} = n - \frac{m}{2}$$

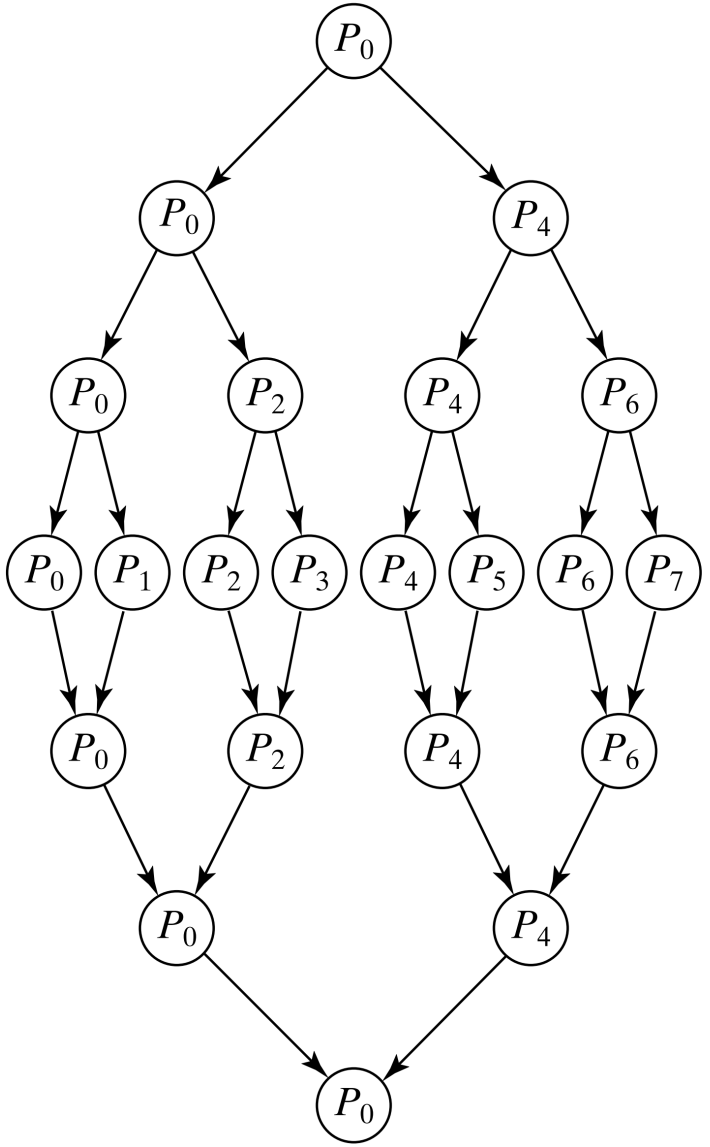
steps to merge all sublists (two by two)

Since in total there are log(n) merging steps, this corresponds to a time complexity of O(n log(n))

Parallel Mergesort

The idea is to take advantage of the tree structure of the algorithm to assign work to processes





Quicksort

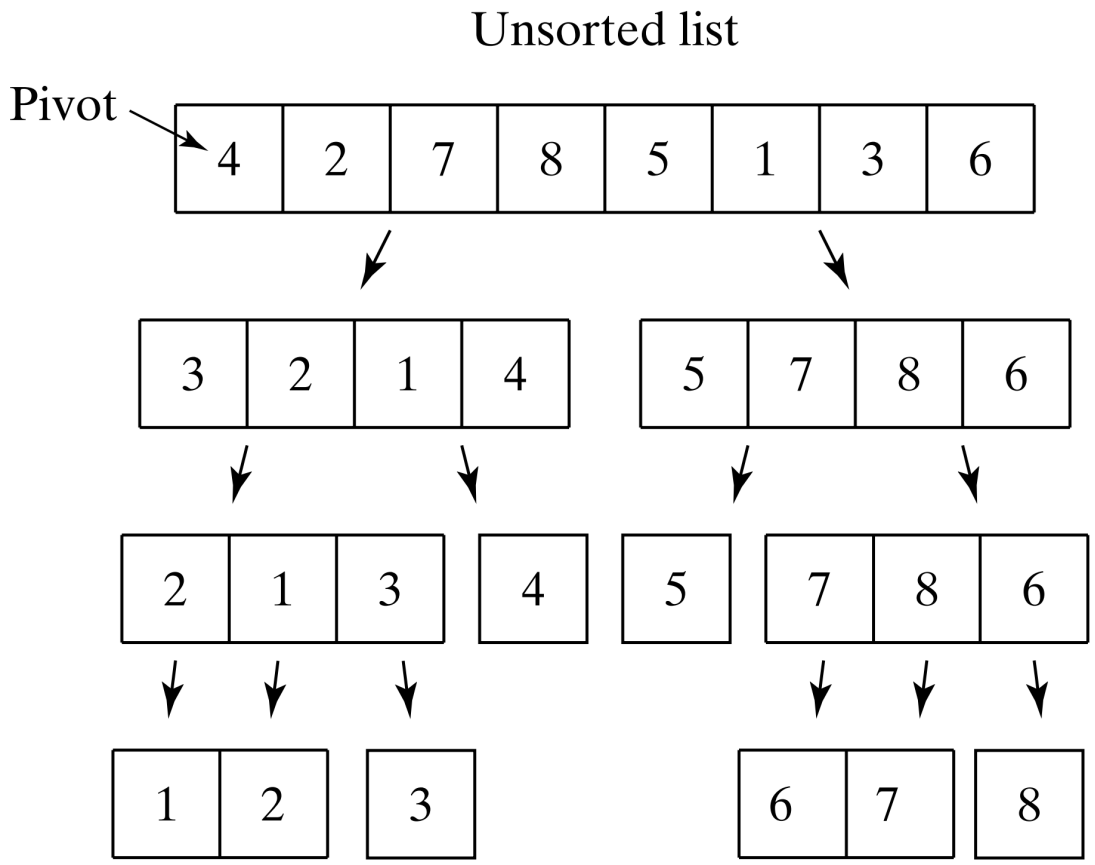
```
quicksort(list, start, end)
{
  if (start < end) {
    partition(list, start, end, pivot)
    quicksort(list, start, pivot-1);
    quicksort(list, pivot+1, end);
  }
}
```

The initial unsorted list is first divided into two sublists in such a way that all elements in the first sublist are smaller than all the elements in the second sublist. This is achieved by selecting one element, called a pivot, against which every other element is compared.

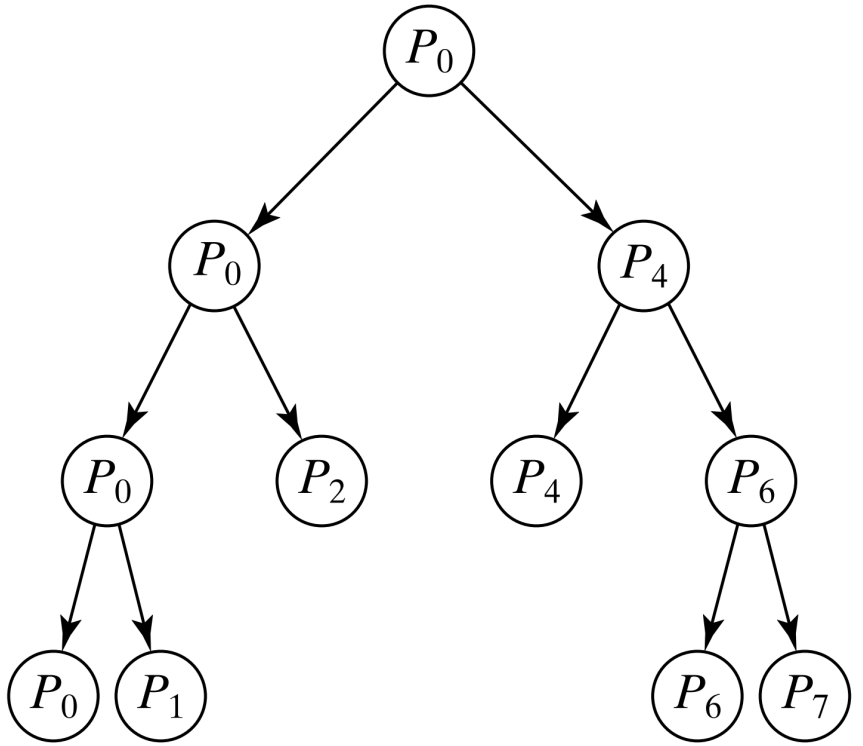
Each sublist is then applied the same division method until individual elements are obtained. With proper ordering of the sublists, the final sorted list is then obtained.

On average, the quicksort algorithm shows a time complexity of O(n log(n)) and, in the worst case, it shows a time complexity of $O(n^2)$

Quicksort



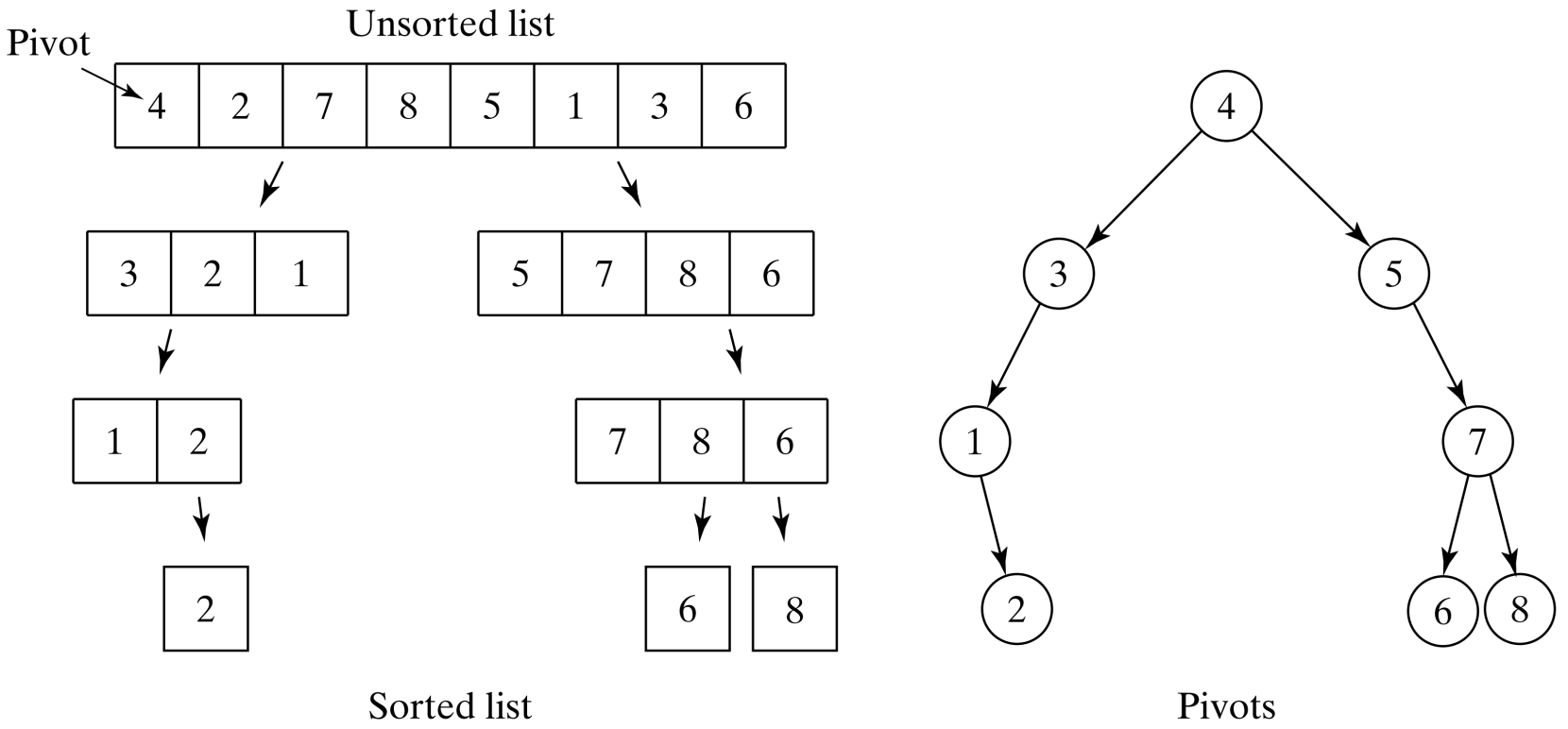
Sorted list



Process allocation

Parallel Quicksort

- As for mergesort, the idea is to take advantage of the tree structure of the algorithm to assign work to processes



Parallel Quicksort

Allocating processes in a tree structure leads to two fundamental problems:

- In general, the partition tree is not perfectly balanced (selecting good pivot candidates is crucial for efficiency)
- The process of assigning work to processes seriously limits the efficient usage of the available processes (the initial partition only involves one process, then the second partition involves two processes, then four processes, and so on)

Again, if we ignore communication time and consider that pivot selection is ideal, i.e., creating sublists of equal size, then it takes

$$\sum_{i=1}^{\log(n)} \frac{n}{2^i} \approx 2n$$

steps to obtain the final sorted list in a parallel implementation, which corresponds to a time complexity of $O(n)$. The worst-case pivot selection degenerates to a time complexity of $O(n^2)$

External Sorting

External Sorting

- All the algorithms we have examined require that the input fit into main memory. There are, however, applications where the input is much too large to fit into memory.

- Most of the internal sorting algorithms take advantage of the fact that memory is directly addressable
  - If the input is on a tape, then all these operations lose their efficiency, since elements on a tape can only be accessed sequentially. Even if the data are on a disk, there is still a practical loss of efficiency because of the delay required to spin the disk and move the disk head

Models for External Sorting

- Let us assume the data is on a tape drive [sequential access]
- We will assume that we have at least three tape drives to perform the sorting
- We need two drives to do an efficient sort; the third drive simplifies matters
- If we have only one drive we cannot do better than $O(n^2)$ in practice

Basic External Sort

- The basic external sorting algorithm uses the merging algorithm from mergesort. Suppose we have four tapes, T_{a1}, T_{a2}, T_{b1}, T_{b2}
- Depending on the point in the algorithm, the a and b tapes are either input tapes or output tapes
- Suppose the data are initially on T_{a1}. Suppose further that the internal memory can hold (and sort) M records at a time

Basic External Sort

T_{a1}	81	94	11	96	12	35	17	99	28	58	41	75	15
T_{a2}													
T_{b1}													
T_{b2}													

Read M records at a time from the input tape (suppose M=3)

Sort the records internally, and then write the sorted records alternately to

T_{b1} and T_{b2}

We will call each set of sorted records a run. When this is done, we rewind all the tapes

Basic External Sort

T_{a1}							
T_{a2}							
T_{b1}	11	81	94	17	28	99	15
T_{b2}	12	35	96	41	58	75	

Now T_{b1} and T_{b2} contain a group of runs. We take the first run from each tape and merge them, writing the result onto T_{a1}

- Recall that merging two sorted lists is simple; we need almost no memory, since the merge is performed as T_{b1} and T_{b2} advance

Then we take the next run from each tape, merge these, and write the result to T_{a2}

Basic External Sort

T_{a1}	11	12	35	81	94	96	15
T_{a2}	17	28	41	58	75	99	
T_{b1}							
T_{b2}							

We rewind all four tapes and repeat the same steps, this time using the **a** tapes as input and the **b** tapes as output

Basic External Sort

T_{a1}													
T_{a2}													
T_{b1}	11	12	17	28	35	41	58	75	81	94	96	99	
T_{b2}	15												

T_{a1}	11	12	15	17	28	35	41	58	75	81	94	96	99
T_{a2}													
T_{b1}													
T_{b2}													

We rewind all four tapes and repeat the same steps, this time using the **a** tapes as input and the **b** tapes as output

Multiway Merge

- If we have extra tapes, then we can expect to reduce the number of passes required to sort our input. We do this by extending the basic (two-way) merge to a k-way merge

- Merging two runs is done by winding each input tape to the beginning of each run. Then the smaller element is found, placed on an output tape, and the appropriate input tape is advanced

- If there are k input tapes, this strategy works the same way, the only difference being that it is slightly more complicated to find the smallest of the k elements

Multiway Merge

T_{a1}						
T_{a2}						
T_{a3}						
T_{b1}	11	81	94	41	58	75
T_{b2}	12	35	96	15		
T_{b3}	17	28	99			

- We can find the smallest of these elements by using a priority queue. Obtain the next element to write on the output tape. The appropriate input tape is advanced, and if the run on the input tape is not yet completed, we insert the new element into the priority queue.

Multiway Merge

T_{a1}	11	12	17	28	35	81	94	96	99
T_{a2}	15	41	58	75					
T_{a3}									
T_{b1}									
T_{b2}									
T_{b3}									

T_{a1}													
T_{a2}													
T_{a3}													
T_{b1}	11	12	15	17	28	35	41	58	75	81	94	96	99
T_{b2}													
T_{b3}													

We then need two more passes of three-way merging to complete the sort